

ICST Tool Competition 2026 – UAV Testing Track

Erdem Uysal
University of Bern
Switzerland

Gregory Loubet-Bonino
University of Bern
Switzerland

Ali Javadi
University of Bern
Switzerland

Aryan Prakash
University of Bern
Switzerland

Aren A. Babikian
University of Toronto
Canada

Dmytro Humeniuk
Polytechnique Montréal
Canada

Sajad Khatiri
University of Bern
Switzerland

Sebastiano Panichella
University of Bern & AI4I
Switzerland & Italy

Abstract—Unmanned Aerial Vehicles (UAVs) are complex, safety-critical cyber-physical systems for which simulation-based testing plays a vital role in validating reliable and safe behavior. Despite its widely acknowledged importance, UAV testing remains an underexplored research area with considerable growth potential. The UAV Testing Competition aims to foster greater involvement from the software testing community and to establish cyber-physical systems testing as an important and evolving research field. The latest edition of the competition was jointly organized by SBFT and ICST and introduced wind disturbances into the simulator to present environmental challenges. In this edition, we received and evaluated a single submission for the ICST track. The submitted tool's test generation capabilities were assessed across multiple case studies and compared against the baseline tool, Surrealist. The evaluation focused on both the diversity of generated test cases and the effectiveness in detecting failures in PX4-Autopilot.

Index Terms—Tool Competition, Software Testing, Test Case Generation, Unmanned Aerial Vehicles, Search Based Software Engineering

I. INTRODUCTION

Simulation-based testing plays a vital role in guaranteeing the safety of autonomous unmanned aerial vehicles (UAVs). Over time, support for UAV developers has expanded through open-access software and hardware projects, including autopilot support from PX4 [12] and Ardupilot. However, despite the critical need for systematically testing these complex and automated systems to ensure their safe operation in the real world, investment in this area has remained relatively limited.

Previous studies have demonstrated that many UAV bugs could potentially be detected before field testing if effective simulation-based testing is implemented [14]. However, the engineering complexity of UAVs and the physical testing environment, and the challenges of creating realistic simulations that accurately capture the same bugs pose significant obstacles [1], [6], [7].

In this context, the UAV Testing Competition offers a simple platform for software testing researchers to ease their entry into the UAV domain. The competition aims to motivate them to apply search-based test generation techniques in a new interesting domain: *autonomous vision-based UAV navigation systems*.

Inspired by previous testing tool competitions [4], [13], we invited researchers to participate in the competition with

their system-level test generation tools for UAVs and compete against the state-of-the-art approach [6].

Building on the strong interest generated by the inaugural UAV Testing Competition in 2024 [8] and incorporating feedback and insights from the second edition in 2025 [10], we made several improvements to the infrastructure and introduced environmental challenges to enhance this year's competition. This 2026 edition was jointly organized by the SBFT and ICST communities [11]. One tool was submitted to ICST 2026 (described in this paper). SBFT 2026 submissions are detailed in the corresponding competition report [15].

II. COMPETITION DESIGN

The objective of the competitors is to develop a test generator for our UAV system under test, a vision-based autonomous flight system known as PX4. Participants must submit a robust test generation tool capable of producing diverse and effective test suites to identify potential vulnerabilities in the system.

This process involves adjusting obstacle sizes, weather condition (eg. wind,...) and placements within the test environment, aiming to either cause the UAV to crash or take an unsafe path, as illustrated in Figure 1. The effectiveness of the generated tests is evaluated based on the minimum distance of the flying drone to the obstacles.

To ensure a fair comparison among competing tools and facilitate their development, we have provided participants with an open-source, extensible test infrastructure [7]:

<https://github.com/skhatiri/UAV-Testing-Competition>

A. Benchmarking Platform

The software under test is PX4 [12], a vision-based UAV autopilot system that has been extensively analyzed in previous research [3], [5], [6]. The testing process is streamlined by Aerialist [7], a UAV test bench built on top of PX4.

1) *PX4 Platform*: PX4 is a widely adopted open-source autopilot software stack in the UAV industry, supporting various vehicle types, from quadcopters to VTOL aircraft. It provides essential functionalities such as navigation, stabilization, and autonomous mission planning while maintaining compatibility with multiple hardware platforms. A key component of this ecosystem is the PX4 Avoidance module, which enables

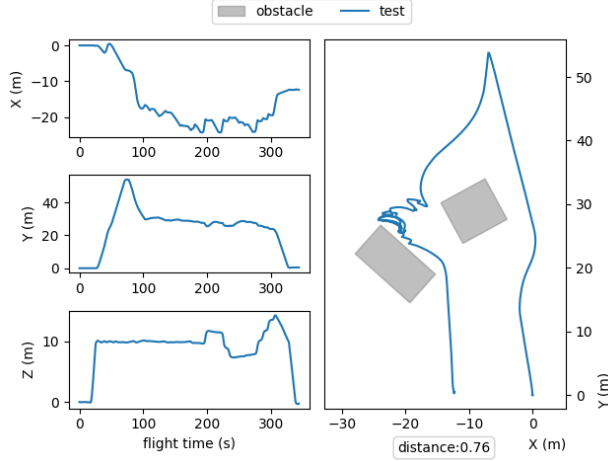


Fig. 1: A failing test case

UAVs to detect and avoid obstacles autonomously—critical for ensuring safe operation in complex environments. PX4 also logs extensive operational data and telemetry, including GPS coordinates, sensor readings, and flight modes, facilitating in-depth flight analysis. Additionally, it supports multiple software-in-the-loop (SITL) simulation environments, such as Gazebo, providing a safe and controlled setting for developing and testing innovative UAV control systems, including mission planning and obstacle avoidance.

2) *Aerialist*: Aerialist¹ (unmanned AERIAL vehicle teST bench) is an advanced test bench for UAV software that fully automates the UAV testing process. It streamlines key steps, including setting up the test environment, compiling and running the UAV firmware, configuring the simulator with appropriate world properties, linking the simulated UAV to the firmware, applying necessary UAV settings, scheduling and executing runtime commands, monitoring the UAV for potential issues, and extracting flight logs upon test completion [7]. By using Aerialist, participants benefit from a user-friendly platform that simplifies UAV test execution, allowing them to focus mainly on their test generation approaches.

Aerialist defines a UAV test case using a structured set of test properties and represents it in a YAML format. This structure encompasses the *drone* properties (such as software configurations and mission plans), *simulation* properties (including the simulator, environment, and obstacles), and the *commands* issued to the drone at runtime. Additionally, Aerialist facilitates large-scale experimentation by enabling test execution deployment within a Kubernetes cluster.

B. Rules and Restrictions

Competition participants were tasked with developing a test generator capable of producing challenging test cases for a given case study. Each case study consists of a basic Aerialist test description in YAML format, including predefined drone

configurations, a mission plan, and simulation environment settings without obstacles. Test generators can only modify the simulation environment by introducing up to three box-shaped obstacles (for simplicity). Obstacles are defined by their dimensions (length, width, height) in meters, position in the simulation environment (x, y, z), and rotation angle (r) in degrees.

The drone is expected to navigate safely through various environments, avoiding obstacles along its path while successfully completing the mission plan. For a given case study, the objective is to generate alternative variations of the surrounding environment (i.e., obstacles) to increase the challenge for the drone's navigation. The introduced obstacles may push the UAV to fly dangerously close—within the 1.5m safety threshold [6]—while still completing the mission, or even result in a crash. Such failing tests highlight potential vulnerabilities that require further investigation by UAV developers (as demonstrated in Figure 1).

Participants are expected to employ search-based methods to identify challenging obstacle configurations. The generated test cases, structured according to Aerialist's modeling framework, must adhere to these considerations:

- Obstacle configurations must ensure the mission remains physically feasible. Test cases that make navigation impossible (e.g., by placing a long wall across the path) without violating safety constraints are invalid.
- All obstacles must fit within the specified rectangular area in the case study.
- A maximum of three obstacles can be placed in the environment (one obstacle less than the previous year to make it more challenging). They must be positioned directly on the ground ($z = 0$), exceed the UAV's flight height ($h > 10m$), and must not overlap.

III. EXPERIMENTS AND RESULTS

A. Competing Tools

One test generation tools named *TQLD* [16], was submitted to the competition. Additionally, we used the state-of-the-art UAV test generation approach, *Surrealist* [6], as the baseline.

TLQD [16] is a Quality-Diversity (QD) based test generator employing a two-level strategy with behavior-guided search. It combines broad global exploration with local search intensification to identify high-severity failures. Additionally, a feasibility repair operator ensures environmental and mission consistency. This dual-layer approach effectively balances high test diversity with maximum failure severity.

Surrealist [6] employs an iterative adaptive greedy search approach to generate test cases that maximize a difficulty measure defined based on the drone's minimum distance to the obstacles.

B. Evaluation Process

The tools were evaluated in two stages: generating a test suite for each case study within a 100-evaluation budget, then assessing and scoring the results.

¹<https://github.com/skhatiri/Aerialist>

1) Test Generation Phase:

a) *Case Studies:* We used four case studies (CS) of increasing difficulty to evaluate the test generation approaches. Adapted from the most challenging cases in the previous edition [8], they vary by number of waypoints (3,4), overlap with obstacle areas, and mission plan shape (rectangular, triangular). Figure 1 shows a sample flight with four waypoints, a rectangular shape, and two obstacles.

b) *Tool Execution:* All submitted tools were executed in a hybrid setup, with the tool itself running directly on a server with 16 vCPUs and 32 GB RAM, and the simulation running in a Docker container. During evaluation, Kubernetes was used to orchestrate the simulations and ensure consistency across tools. For each case study, the simulation budget for each tool was set to 100, representing the maximum number of test cases allowed for simulation. To ensure consistency, the test cases were executed in separate, isolated containers with fixed resource allocations (6 vCPUs, 4 GB RAM). For each simulation, a timeout of 500 seconds was allocated, after which the simulation was terminated, and logs were extracted. After each test case execution, the tools are expected to generate a test suite containing the failing tests found during the execution and ordered based on their relevance and criticality. These failing tests were then used for the final evaluation.

2) *Test Suite Evaluation Phase:* Due to time and computational constraints, evaluations were limited to 20 generated tests per tool for each case study, selected sequentially from the test generator output. These tests were first checked for compliance with competition rules (e.g., obstacle size and placement). Valid tests were then executed three times to address non-determinism, with each run scored using the minimum distance between the UAV trajectory and obstacles (*min_dist*) as per the Equation 1 shown below.

$$point(sim) = \begin{cases} 5, & \text{if } min_dist(sim) < 0.25m \\ 2, & \text{if } 0.25m \leq min_dist(sim) < 1m \\ 1, & \text{if } 1m \leq min_dist(sim) < 1.5m \\ 0, & \text{if } min_dist(sim) \geq 1.5m \end{cases} \quad (1)$$

The *test_score* for each test case was calculated using its average point (*avg_point*), as shown in Formula 2. To encourage scenarios with fewer obstacles—a more challenging task—we penalized test cases with more obstacles and longer test execution times (*avg_time*) in minutes.

$$test_score(t) = \frac{avg_point(t) \times 10}{\#obst(t)^2 \times avg_time(t)} \quad (2)$$

The average score *Failure_score* of a test suite *s* was calculated as the sum of all the test scores (excluding duplicates):

$$F_s = \sum_{t \in s} test_score(t) \quad (3)$$

To quantify the *input diversity* of a generated test suite, we define a geometry-based diversity metric that captures both the spatial coverage of test scenarios and the variability in their

geometric extent. Let $\mathcal{G} = \{G_1, G_2, \dots, G_n\}$ be a set of $n \geq 2$ test geometries, where each geometry represents an obstacle that is part of the test scenario, and let $A(G_i)$ denote the area of geometry G_i .

First, we measure the overall spatial coverage of the test by computing the ratio between the area of the union of all geometries and the average area of individual geometries:

$$D_{union} = \frac{A(\bigcup_{i=1}^n G_i)}{\frac{1}{n} \sum_{i=1}^n A(G_i)}.$$

This term penalizes redundant test scenarios with overlapping obstacles while rewarding tests that collectively cover a larger portion of the test area.

Second, to account for structural variability among test scenarios, we compute the coefficient of variation of the geometry areas:

$$CV = \frac{\sqrt{\frac{1}{n-1} \sum_{i=1}^n (A(G_i) - \bar{A})^2}}{\bar{A}}, \quad \text{where } \bar{A} = \frac{1}{n} \sum_{i=1}^n A(G_i).$$

Intuitively, we want to assign increased reward to tests that contain obstacles with varying (diverse) areas. Finally, the overall diversity of the test suite is defined as:

$$D_{in} = D_{union} \cdot (1 + CV).$$

If all obstacles present in all tests of a test suite have the same area, the diversity metric *D* will equal the initially computed *D_{union}*. Variations in obstacle area within a test suite lead to an increased coefficient of variation *CV* and to a higher final diversity metric *D_{in}*, thus promoting diversity in obstacle areas. Apart from the test suite *input diversity*, we also compute the test suite *output diversity*, characterized by the difference in the test subject trajectories across the tests for the same mission. We compute it as follows. We assume having $m > 1$ generated test scenarios.

We re-run the scenario evaluation 3 times for each test *j*, producing trajectories:

$$\mathcal{T}_j = \{\tau_{j,1}, \tau_{j,2}, \tau_{j,3}\}, \quad j = 1, \dots, m$$

For each test we compute the averaged trajectory $\bar{\tau}_j$

$$\bar{\tau}_j = \text{Avg}(\mathcal{T}_j)$$

As a result, we obtain *m* averaged trajectories:

$$\bar{\mathcal{T}} = \{\bar{\tau}_1, \bar{\tau}_2, \dots, \bar{\tau}_m\}$$

We then compute the nearest neighbor dynamic time warping [1] (DTW) distance δ_j for each test:

$$\delta_j = \min_{\substack{k=1 \\ k \neq j}}^m \text{DTW}(\bar{\tau}_j, \bar{\tau}_k)$$

We then average the minimal nearest neighbor distance across the tests, resulting in a final *output diversity* metric:

$$D_{out} = \frac{1}{m} \sum_{j=1}^m \delta_j$$

TABLE I: Ranking of the tools (TLQD vs Surrealist)

Tool Name	#Obst.	CS2				CS3				CS4				CS5				Average			Final Rank
		#tests	F_s	D_{in}	D_{out}	#tests	F_s	D_{in}	D_{out}	#tests	F_s	D_{in}	D_{out}	#tests	F_s	D_{in}	D_{out}	F_s	D_{in}	D_{out}	
TLQD	2	100 (10)	0.574	3.36	3.29	100 (7)	1.270	4.77	5.96	100 (10)	0.131	5.24	2.54	100 (10)	0.954	2.07	5.84	0.732	3.86	4.41	2
Surrealist	2	80 (17)	18.25	1.29	1.52	108 (19)	26.28	1.62	14.74	13 (4)	2.44	1.34	3.60	79 (0)	0.00	1.44	7.00	11.74	1.43	6.72	1

C. Results and Ranking

Table I summarizes the evaluation metrics. For each case study (CS), we report the total test cases (100 for *TLQD*), failed cases (in parenthesis), failure (F_s), and diversity scores (D_{in} , D_{out}). Ranking is calculated via a weighted sum ($0.75 \times \sum R_F + 0.25 \times \sum R_D$). Detailed flight plots and artifacts are available at [9].

Overall, *Surrealist* led in failure severity (F_s), securing the first rank in all missions. In contrast, *TLQD* demonstrated superior exploration capabilities, achieving the highest *Input Diversity* (D_{in}) across all scenarios. Notably, *TLQD* reached a D_{in} of **5.24** in *CS4*, significantly outperforming *Surrealist*. While its F_s remained lower, *TLQD* showed strong *Output Diversity* (D_{out}) in *CS2* and *CS3*, indicating high variance in generated trajectories. *CS3* was the most productive for *TLQD* regarding failure detection ($F_s = 1.27$). In the final ranking, *Surrealist* placed first due to the competition's high weighting on failure severity. Nevertheless, *TLQD*'s results confirm its dominance in generating geographically diverse and varied test environments.

IV. CONCLUSION AND FINAL REMARKS

This year marked the third edition of the UAV Testing Competition, jointly organized at ICST and SBFT 2026. We evaluated and compared one tool at ICST using *Surrealist* as a baseline, with the proposed tool emerging as the top performer. This edition introduced wind disturbances in the simulator to increase testing challenges and a new input diversity metric based on obstacle shape similarity. The 2026 competition was limited to box-shaped obstacles. Future editions will introduce new metrics [2] and expand to diverse obstacle types, such as trees and buildings, as well as environmental factors.

ACKNOWLEDGMENTS

We thank all the participants for their contribution to the competition. We thank the Horizon 2020 (EU Commission) support for the project InnoGuard, Marie Skłodowska-Curie Actions Doctoral Networks (HORIZON-MSCA-2023-DN), the Hasler Foundation for the project "Safe2Fly" (No. 2025-02-27-311), and the SNSF for the project entitled "SwarmOps: Human-sensing based MLOps for Collaborative Cyber-physical systems" (Project No. 200021_219732).

REFERENCES

- [1] ARYAN, P., AND PANICHELLA, S. Physics-informed machine learning for precision unmanned aerial vehicle control: Adaptive transformers with safety guarantees. *Eng. Appl. Artif. Intell.* 172 (2026), 114379.
- [2] BIRCHLER, C., MOHAMMED, T. K., RANI, P., NECHITA, T., KEHRER, T., AND PANICHELLA, S. How does simulation-based testing for self-driving cars match human perception? *Proc. ACM Softw. Eng.* 1, FSE (2024), 929–950.
- [3] DI SORBO, A., ZAMPETTI, F., VISAGGIO, C. A., DI PENTA, M., AND PANICHELLA, S. Automated Identification and Qualitative Characterization of Safety Concerns Reported in UAV Software Platforms. *ACM Trans. Softw. Eng. Methodol.* (Sept. 2022).
- [4] GAMBI, A., JAHANGIROVA, G., AND RICCIO, V. Message from sbft 2024 program chairs. In *2024 IEEE/ACM International Workshop on Search-Based and Fuzz Testing (SBFT)* (2024), pp. ix–ix.
- [5] KHATIRI, S., AMIN, F. M., PANICHELLA, S., AND TONELLA, P. When uncertainty leads to unsafety: Empirical insights into the role of uncertainty in unmanned aerial vehicle safety. *Empir. Softw. Eng.* 30, 6 (2025), 166.
- [6] KHATIRI, S., PANICHELLA, S., AND TONELLA, P. Simulation-based test case generation for unmanned aerial vehicles in the neighborhood of real flights. In *International Conference on Software Testing, Verification and Validation* (2023).
- [7] KHATIRI, S., PANICHELLA, S., AND TONELLA, P. Simulation-based testing of unmanned aerial vehicles with aerialist. In *International Conference on Software Engineering (ICSE)* (2024).
- [8] KHATIRI, S., SAURABH, P., ZIMMERMANN, T., MUNASINGHE, C., BIRCHLER, C., AND PANICHELLA, S. "sbft tool competition on testing unmanned aerial vehicles", 2023.
- [9] KHATIRI, S., ZOHODINASAB, T., PRASUN, S., HUMENIUK, D., AND PANICHELLA, S. ICST 2025 uav testing competition: Evaluation artifacts". <https://doi.org/10.5281/zenodo.14714469>, 2025.
- [10] KHATIRI, S., ZOHODINASAB, T., SAURABH, P., HUMENIUK, D., AND PANICHELLA, S. ICST tool competition 2025 - uav testing track. In *IEEE/ACM International Conference on Software Testing, Verification and Validation, ICST 2025* (2025).
- [11] KHATIRI, S., ZOHODINASAB, T., SAURABH, P., HUMENIUK, D., AND PANICHELLA, S. SBFT tool competition 2025 - uav testing track. In *IEEE/ACM International Workshop on Search-Based and Fuzz Testing, SBFT@ICSE 2025* (2025).
- [12] MEIER, L., HONEGGER, D., AND POLLEFEYS, M. Px4: A node-based multithreaded open source robotics framework for deeply embedded platforms. In *international conference on robotics and automation* (2015), IEEE, pp. 6235–6240.
- [13] PANICHELLA, S., GAMBI, A., ZAMPETTI, F., AND RICCIO, V. SBST tool competition 2021. In *International Workshop on Search-Based Software Testing* (2021), IEEE, pp. 20–27.
- [14] TIMPERLEY, C. S., AFZAL, A., KATZ, D. S., HERNANDEZ, J. M., AND LE GOUES, C. Crashing simulated planes is cheap: Can simulation detect robotics bugs early? In *2018 IEEE 11th International Conference on Software Testing, Verification and Validation (ICST)* (2018), IEEE, pp. 331–342.
- [15] UYSAL, E., JAVADI, A., ARYAN, P., BABIKIAN, A., HUMENIUK, D., MAZRAEHKHATIRI, S., AND PANICHELLA, S. SBFT tool competition 2026 – uav testing track. In *International Workshop on Search-Based and Fuzz Testing, SBFT@ICSE 2026* (2026).
- [16] ZHANG, H., ZHOU, S., QIN, H., AND XIANG, Y. TLQD at the ICST 2026 tool competition – UAV testing track. In *Proceedings of the IEEE International Conference on Software Testing, Verification and Validation (ICST) Tool Track* (2026). Competition Report.